BugTrail

A bug report goes in. A named pull request, its author, the suspect diff, the owning team, and a proven failing regression test come out.

Team Documentation | Python 3.9+ · stdlib only | No network · no credentials | 15 tests passing | Attribution mechanically verified

> A bug report goes in. A named pull request, its author, the suspect diff, the owning team, and a **proven** failing regression test come out.

Repository: `Monish-WBD/marrow-bugtrail-fixture` (public) **Stack:** Python 3.9+ standard library only, plus `git`. No dependencies, no network, no credentials.

Table of contents

---

1. The problem

When a bug is filed, someone has to answer two questions:

> **Which change caused this?** and **who should look at it?**

CodeSage already answers the first half. It posts an AI triage comment suggesting priority, severity, component, team, and a *starting-point file*.

What it does **not** do is close the loop to source control. An engineer still has to:

1. Run `git log` on the suggested file
2. Read through recent commits
3. Work out which PR each commit came from
4. Check who authored it
5. Decide which one is plausibly related
6. Write a test that reproduces the bug

That is **10–20 minutes of mechanical archaeology per bug**, repeated for every bug, on every platform.

BugTrail automates exactly that gap. It *consumes* CodeSage's output rather than replacing it.

---

2. What BugTrail does

Does

CAPABILITY	DETAIL
Parse triage input	Reads a CodeSage comment deterministically; handles Jira's ADF format
Walk history	Follows the seed file through renames, plus its module siblings
Map commit → PR	Handles squash merges, merge commits, and direct pushes with no PR
Exclude the impossible	Bots, reverts, generated sources, anything landing after the report
Rank and explain	Four weighted signals, every score explained in plain English
Resolve ownership	CODEOWNERS lookup, and flags disagreement with CodeSage's suggested team
Draft a regression test	Correct path, framework, class name and symbol, derived from the suspect diff
Prove the attribution	Compiles the test at two revisions to confirm behaviour changed at that commit

Does not

- Re-implement CodeSage's root-cause analysis or triage field suggestions
- Claim certainty — output is *"likely related changes"*, never *"X broke this"*
- Write the assertion inside a drafted test (see §7)
- Talk to Jira or GitHub **yet** (see §10)

---

3. Quick start

```bash
cd /Users/monk/Documents/Project/Marrow
./demo.sh
```

Runtime: ~3 seconds. This is the whole story end to end.

```bash
./demo.sh            # Android case  (SYN-002) — highest confidence
./demo.sh SYN-001    # iOS case      — the one with the compiled proof
./demo.sh SYN-003    # renamed-file case — where naive git log gets it wrong
```

What you will see

Step 1 — triage comment in, suspect PR out:

```
================================================================================
BUG SYN-002
Skip Intro marker not propagated to overlay for preroll
================================================================================

Reported        2026-07-02 04:00 UTC
Platform        android
CodeSage        priority P1 | severity S2 | component Android Player – Timeline
Suggested team  MARROW-Android
Regression      yes (per CodeSage summary)

SEED FILE (CodeSage starting point)
  android/.../adskip/DefaultAdSkipManager.kt
  CODEOWNERS: @marrow-android-player   (agrees with CodeSage's suggested team)

--------------------------------------------------------------------------------
LIKELY RELATED CHANGES                                  confidence: High (0.82)
--------------------------------------------------------------------------------
```

```
1.  PR #11   "Fix preroll marker propagation to overlays (#11)"
    commit    f10254a  (squash merge)
    author    Bob Sharma <bob@example.com>
    landed    2026-07-01 13:40
    score     0.82   (4 line(s) changed)
    why
              - modifies the file CodeSage pointed to
              - changed code mentions 'skip', 'marker', 'overlay', 'preroll'
              - landed the same day the bug was reported

    suspect change
      -    fun markersForOverlay(): List<TimelineMarker> = processor.skippableMarkers()
      +    fun markersForOverlay(): List<TimelineMarker> =
      +        processor.skippableMarkers().filterNot { it.isPreroll }
```

Step 2 — proving the drafted test isolates that PR:

```
at HEAD (bug present, expect FAIL)
  FAIL  a skippable preroll was reported as not skippable
  => as expected: the test catches the bug

at 5ccb5dc^ (before the suspect PR, expect PASS)
  PASS  a skippable preroll remains skippable on the ad-free tier
  => as expected: the behaviour was intact here

CONFIRMED: behaviour changed at the suspect commit. Attribution holds.
```

---

4. How to test everything

All commands run from the repository root. Nothing needs a network.

4.1 Full report

```bash
python3 tools/bugtrail/cli.py --comment fixtures/triage/SYN-002.txt --report
```

Expect: bug details, ranked suspects with authors and reasons, suspect diff, CODEOWNERS, drafted test.

4.2 Prove the regression test isolates the suspect

```bash
tools/bugtrail/verify/verify.sh
```

Expect: `FAIL` at HEAD, `PASS` before the suspect commit, ending `CONFIRMED`. Requires `swiftc`.

4.3 Score the ranker against known culprits

```bash
python3 tools/bugtrail/cli.py --eval
```

```
bug       expected  predicted    P@1   P@3
SYN-001   #5        5, 2, 4      yes   yes
SYN-002   #11       11, 3        yes   yes
SYN-003   #4        4, 5, 2      yes   yes
precision@1: 3/3 (100%)   precision@3: 3/3 (100%)
```

4.4 Derive ground truth from git history, then score against it

```bash
python3 tools/bugtrail/mine_szz.py --repo . --scan 200 --out fixtures/mined-ground-truth.json
python3 tools/bugtrail/cli.py --eval --manifest fixtures/mined-ground-truth.json --repo .
```

Expect: 2 bugs derived, `2/2`. This proves the *mechanism*, not accuracy — see [§6](#6-what-we-proved).

4.5 Unit tests

```bash
python3 -m unittest discover -s tools/bugtrail/tests -v
```

Expect: 15 passing, including the CodeSage format contract test.

4.6 Machine-readable output

```bash
python3 tools/bugtrail/cli.py --comment fixtures/triage/SYN-001.txt --json
```

Expect: JSON with suspects, commit SHAs, authors, scores, reasons. This is what a Jira adapter will consume.

4.7 The Cursor skill

In a Cursor chat inside this repo:

```
use bugtrail on fixtures/triage/SYN-001.txt
```

Expect: the agent reads `.cursor/skills/bugtrail/SKILL.md`, runs the engine, fills the test assertion, verifies it, and composes the report.

4.8 Regenerating the fixture from scratch

```bash
git reset --hard origin/main
./tools/seed_fixture.sh
```

> ⚠️ Do **not** run `git clean -fd` — it deletes untracked work.

4.9 Useful flags

FLAG	PURPOSE
`--repo <path>`	Analyse a different repository
`--history-limit <n>`	Commits inspected per file; lower on large repos
`--no-module-expansion`	Consider only the seed file, not siblings
`--reported-at <iso>`	Override the report timestamp
`--manifest <path>`	Score against a different bug set
`--config <path>`	Alternative tuning

---

5. How it works

5.1 Pipeline

5.2 Commit → PR mapping

Three cases, all handled:

MERGE STYLE	COMMIT SUBJECT	HOW IT IS RESOLVED
Squash merge	`Fix preroll propagation (#11)`	Regex on the subject
Merge commit	`Merge pull request #10 from …`	Regex on the subject
Merged *via* a merge commit	`refactor: rename …` (no number)	Walk `--ancestry-path` forward to the earliest containing merge
Pushed straight to main	`hotfix: widen tolerance`	No PR — reported as a bare commit

5.3 Scoring

SIGNAL	WEIGHT	WHY
Recency	0.35	Exponential decay toward the report date; regressions are usually recent
Keyword overlap	0.35	Matched against **code lines only**, so comments cannot inflate it
Seed-file match	0.20	The file CodeSage named outranks a module sibling
Substantiveness	0.10	Did the change touch code at all?

Then **penalties**: comment-only and rename-only changes are multiplied by `1 - cosmeticPenalty` (default `0.75`).

> **This penalty does the real work.** In the fixture, a docs-only commit lands on the culprit file *one day later* than the true cause. Pure recency ranks it first and gets the wrong answer.

Finally: collapse to one entry per PR, and if the top score is below `minConfidence`, report **no suspect** rather than guessing.

5.4 Configuration

Everything is in `tools/bugtrail/config.json` — never in code, so another team adopts this by editing a file:

JSON

```
{
  "historyLimit": 60,
  "expandToModule": true,
  "halfLifeDays": 30,
  "moduleWeight": 0.5,
  "cosmeticPenalty": 0.75,
  "minConfidence": 0.35,
  "maxSuspects": 3,
  "excludeReverts": true,
  "weights": {
    "recency": 0.35,
    "keyword": 0.35,
    "seedFile": 0.20,
    "substantive": 0.10
  },
  "botAuthorPatterns": ["\\[bot\\]", "^dependabot", "^github-actions", "^svc-"],
  "generatedPathGlobs": ["*/Generated/*", "*/generated/*"],
  "generatedMarkers": ["GENERATED CODE DO NOT MODIFY", "@generated"]
}
```

6. What we proved

6.1 The fixture is adversarial by design

The seeded repository contains deliberate traps, so a naive implementation fails:

TRAP	WHAT IT DEFEATS
Comment-only change to the culprit file, **one day later**	"Most recent commit wins"
A file rename	Diffs that look like whole-file rewrites
A culprit **predating** the rename	Any approach without `git log --follow`
A revert commit	Blaming the person who undid something
A `github-actions[bot]` commit	Blaming automation
A file marked `GENERATED CODE DO NOT MODIFY`	Blaming whoever ran a generator
A hotfix pushed straight to main, no PR	Assuming every commit has a PR

6.2 Results

CHECK	RESULT
Fixture eval	`precision@1: 3/3`, `precision@3: 3/3`
SZZ-derived eval	`2/2` (mechanism proof only — see below)
Unit tests	15 passing
Attribution proof	Test fails at HEAD, passes before suspect commit → `CONFIRMED`

6.3 Read these numbers honestly

> `3/3` **is a regression guard, not evidence of accuracy.** Three cases, hand-built, scored by a ranker tuned against them. It exists so a change to the ranker cannot silently make attribution worse. `2/2` **from SZZ is a mechanism demo.** This repository's history only yields two mineable fixes, and in both the blamed cause is the file's *creation* commit — an artefact of a short history. It shows the pipeline runs; it measures nothing.

A meaningful figure needs a repository with years of history and enough single-file fixes for the sample size to be worth quoting. The harness is ready for that: `--eval --manifest <file> --repo <path>`.

7. Key design decisions

7.1 Keep the model out of the load-bearing path

DETERMINISTIC — REPRODUCIBLE FROM THE REPO ALONE	JUDGEMENT — MODEL OR HUMAN
Which commits touched the file	Filling a regression test's assertion
Which PR each commit came from	Phrasing the summary for humans
Exclusions (bots, reverts, generated)	Deciding whether a suspect is plausible
The scores and their ranking	Explaining *why* a change broke behaviour

Why: any suspect can be re-derived by hand with `git log`, and any score recomputed from the printed reasons. Nothing has to be taken on trust — which is what makes the output survive review.

A side effect worth stating: swapping model or vendor **cannot change which PR is named**.

7.2 Fail closed, never guess

- No `- File:` line in the comment → return `None`, analyse nothing.
- Top score below threshold → report **no suspect**.
- Seed file is generated → refuse to attribute.

> A wrong seed produces *confidently* wrong attribution, which is worse than no answer.

7.3 The drafted test's assertion is deliberately a `TODO`

The scaffold is deterministic: path, framework, class name, and the symbol under test are all derived from the suspect diff. The assertion is not, because writing it requires knowing what the code was *supposed* to do.

> A generator that emitted confident-looking assertions would demo beautifully and mislead on the first real bug.

7.4 Attribution is a suggestion, not a verdict

Every report says *"likely related changes"* and ends with *"ranked suggestions, not attributions of fault."* Attribution errors are cheap; blaming the wrong engineer is not.

---

8. Bugs we hit and fixed

Both only appear on realistic repositories — worth knowing for the Q&A.

8.1 Renames faked keyword matches

Symptom: the rename PR outranked the true culprit.

Cause: git renders a pure rename as a whole-file delete plus add, so the rename commit appeared to mention *every* keyword in the bug report.

Fix: detect renames with `--name-status -M`, run **without** a pathspec (rename detection needs both sides of the diff), and treat them as non-behavioural.

8.2 Historical paths broke diff analysis

Symptom: the true culprit vanished from the results entirely for the renamed-file case.

Cause: `--follow` correctly walked back past the rename, but the engine then diffed that old commit against the file's **current** name — which did not exist yet. Empty diff → looked cosmetic → penalised out.

Fix: parse `--name-status` alongside the log to recover each commit's path *at that commit*.

> This is the more instructive failure: invisible on a repository where nothing has moved, guaranteed on one where things have.

8.3 Timestamp parsing

Real histories mix `+00:00` and `Z` offsets. Python 3.9's `fromisoformat` rejects the latter and crashed the miner. Now handled centrally in `parse_timestamp`.

---

9. Repository layout

```
.
├── demo.sh                          # ← the 3-second end-to-end demo
├── BUGTRAIL.md                      # ← this document
├── docs/BUGTRAIL.pdf                # ← this document, printable
│
├── .cursor/skills/bugtrail/
│   └── SKILL.md                     # Cursor skill wrapping the workflow
│
├── tools/bugtrail/
│   ├── cli.py              (330)    # sources, orchestration, eval
│   ├── archaeology.py      (389)    # git history, PR mapping, exclusions
│   ├── mine_szz.py         (269)    # derive ground truth from history
│   ├── report.py           (171)    # full report rendering
```

```
│   ├── codesage.py          (140)      # ADF flatten + comment parsing
│   ├── testgen.py           (137)      # regression test scaffolding
│   ├── ranking.py           (123)      # keyword extraction + scoring
│   ├── models.py             (82)      # shared data types
│   ├── codeowners.py         (75)      # owner resolution
│   ├── config.json                     # all tuning lives here
│   ├── README.md                       # setup, usage, limitations
│   ├── ARCHITECTURE.md                 # design and data flow
│   ├── tests/test_bugtrail.py (215)    # 15 tests
│   └── verify/
│       ├── verify.sh                   # compiles assertion at two revisions
│       └── main.swift                  # the assertion itself
│
├── fixtures/
│   ├── ground-truth.json               # answer key for the seeded bugs
│   └── triage/SYN-00{1,2,3}.txt        # synthetic CodeSage comments
│
├── tools/docs/md2pdf.py     (330)      # renders this document to PDF
└── tools/seed_fixture.sh               # regenerates the adversarial history
```

Regenerate the PDF after editing this file:

```bash
python3 tools/docs/md2pdf.py BUGTRAIL.md docs/BUGTRAIL.pdf
```

~1,930 lines of Python, standard library only.

---

10. Path to production — Jira

This is the remaining work. The architecture was built for it: the engine never learns where its input came from or where its output goes.

```
TriageSource  →  yields TriageInput
  ├── codesage_source   (built)  reads a comment file
  └── JiraSource        (TODO)   reads issue + CodeSage comment

ResultSink    ←  receives ranked suspects
  ├── ConsoleSink       (built)  renders the report
  └── JiraCommentSink   (TODO)   posts one idempotent comment
```

> Adding Jira is a **new adapter**, not a change to archaeology or ranking.

10.1 What is already done

PIECE	STATUS
ADF (Atlassian Document Format) flattening	✅ `codesage.flatten_adf()` — Jira v3 returns comment bodies as ADF JSON, not text
CodeSage comment parsing	✅ `codesage.parse_adf_comment()`
Structured output for posting	✅ `cli.to_dict()` / `--json`
Report shaped like a Jira comment	✅ `report.render_report()`
Platform → repo routing	✅ derived from the seed file extension

10.2 Step 1 — Trigger

Recommended: a JQL poller. No infrastructure, no admin rights, and it naturally waits for CodeSage to post first.

```sql
project = PLAY
  AND issuetype = Bug
  AND created >= -1h
  AND comment ~ "AI Triage Suggestion"     -- CodeSage has run
  AND NOT comment ~ "BugTrail"             -- we have not
```

OPTION	INFRASTRUCTURE	ADMIN RIGHTS	NOTES
JQL poller ✅	None	None	Idempotent, waits for CodeSage naturally
Jira Automation rule	None	Project admin	Free tier caps at 100 executions/month
Webhook → service	Yes	Site admin	The hardened end state

> **Licensing is not a blocker.** The REST API is available on every Jira plan. Only *native Automation rules* are metered.

10.3 Step 2 — Read path

```python
# jira_source.py  (to build)
issue    = get_jira_issue(key)                    # summary, created, comments
comment  = first(c for c in issue.comments
                 if c.author.accountId == CODESAGE_ACCOUNT_ID)
triage   = parse_adf_comment(comment.body)        # already implemented
```

Identify CodeSage by service account, not by text. The account is `svc-wbdstreaming-play-codesage`; matching on the author id is far more robust than grepping for a marker string. Use JQL to *find* candidates, then verify the author id when fetching.

10.4 Step 3 — Write path

```python
# jira_sink.py  (to build)
MARKER = "<!-- bugtrail:v1 -->"

existing = find_comment_containing(issue, MARKER)
body     = MARKER + render_report_as_adf(result)

if existing:
    update_comment(issue, existing.id, body)   # never duplicate
else:
    add_comment(issue, body)
```

Rules:

- **One comment per issue.** Update in place; a re-run must never post a second comment.
- **Only post above the confidence threshold.** Below it, post nothing.
- **Keep the disclaimer.** Mirror CodeSage's own "please review" convention.
- **Never `@`-mention** an author below high confidence.

10.5 Step 4 — Repository routing

```python
REPOS = {
    "ios":     "/path/to/ios-repo",
    "android": "/path/to/android-repo",
}
repo = REPOS[triage.platform]     # platform derived from the seed file extension
```

Clones must be kept fresh (`git fetch` before analysis) or attribution will silently miss recent PRs.

10.6 Step 5 — Let the agent fill the test assertion

Behind the Cursor skill, gated on human review. Never auto-commit a generated test.

---

11. Production hardening checklist

11.1 Security and IP

REQUIREMENT	STATUS	ACTION
No PII, tokens, or credentials in prompts	⚠️	**Write a sanitiser with its own tests** before any log or stack trace reaches a model
Service account, least privilege	❌	Jira: *add comment* only. GitHub: *read* only. Never a personal token
Secrets in the approved vault	❌	Not env files, never in prompts
Audit trail	❌	Log what was read, what was decided, what was posted
No unvetted dependencies	✅	Standard library only
Nothing internal in public repos	⚠️	Repository is **public**. Every fixture is synthetic and `fixtures/codesage/` is gitignored, but this document does describe CodeSage's behaviour and comment format — worth a review before wider circulation
Human review before code lands	✅	Drafted tests are proposals, never committed

11.2 Reliability

CONCERN	HANDLING
Duplicate comments	Marker + update-in-place
Jira or GitHub unavailable	Bounded retries with backoff; degrade to a partial comment rather than failing loudly
CodeSage format changes	Contract test pinned to the sample comment — CI fails loudly instead of parsing to nothing
Runaway posting	Feature flag / kill switch, no deploy required
Stale clones	`git fetch` before analysis

11.3 Observability and feedback

- **Metrics:** runs, latency, post success rate, error rate, and precision over time.
- **Feedback signal:** a Jira label (`bugtrail-correct` / `bugtrail-wrong`) applied by engineers. This does three jobs at once — collects ground truth continuously, lets you tune ranking weights on real data, and produces the measured-impact number.

11.4 Rollout stages

```
1. Shadow mode      compute and log, post nothing        ← validates safely
2. One project      post to a single project, opt-in
3. Opt-in teams     teams enable it themselves
4. Default on       with a documented kill switch
```

---

12. Known limitations

LIMITATION	IMPACT
Keyword matching is substring-based	`roll` matches `preroll` — loose but harmless
Module expansion covers only the immediate directory	A cause two directories away is missed
PR mapping relies on GitHub message conventions	A repo with different merge messages needs the GitHub API path
Bugs caused by config, content, or backend	No code culprit exists; the confidence threshold prevents inventing one
Precision unmeasured on a large real corpus	See §6.3
Kotlin test written but not executed	No Gradle here; only the Swift case is mechanically confirmed

---

13. Work split and next steps

Suggested split

OWNER	AREA
Dev A	`JiraSource` — issue + CodeSage comment via the approved Atlassian integration
Dev B	`JiraCommentSink` + JQL poller + idempotency marker
Dev C (iOS)	Repo routing and validation against the iOS repository
Dev D (Android)	Same for Android, plus a Gradle target so the Kotlin test is executed too

Priority order

1. **Real eval on a large repository** — the number that makes everything else believable
2. **Jira read path** (shadow mode — log, do not post)
3. **Jira write path** with idempotency
4. **JQL poller**
5. **Sanitiser** with tests, before anything reaches a model
6. **Feedback label** to collect ground truth continuously

The 30-second summary for a reviewer

> CodeSage tells us *where* to look. BugTrail closes the loop to source control: it names the pull requests that likely caused the bug, who authored them, which team owns the file, and it drafts a regression test — then **compiles that test at two revisions to prove the behaviour changed at that commit and nowhere else**. The part that decides *which PR* is fully deterministic, so every result can be reproduced by hand with `git log`. A model is used only where judgement is genuinely required, and never for attribution.